MEKELLE UNIVERSITY

ETHIOPIAN INSTITUTE OF TECHNOGY -MEKELLE(EIT-M)

DEPARTMENT: SOFTWARE ENGINEERING

TITLE: GROUP ASSIGNMENT

COURSE TITLE: DATA STRUCTURE & ALGORITHMS

COURSE CODE: SENG2122

SECTOIN:2

R.NO	NAME	ID NUMBER
1	Mebrihit Gebrekrstos	Ugr/195456/17
2	Tesfaye Tekle	Ugr/196042/17
3	William Muez	Ugr/196099/17
4	Michaele Equbay	Ugr/197290/17
5	Adonay Kiros	Ugr/194421/17
6	Robel Yitebarek	Ugr/195817/17

SUBMITTED TO: INS.HABTAMU

SUBMITTION DATE: 19/08/2018 E.C

Version Control System Simulator

Behavioral Walkthrough — Complete Step-by-Step Demonstration

MAY 4, 2026

Overview

This document provides a complete, step-by-step demonstration of a custom C++ Version Control System (VCS) simulator. Each operation is shown with a terminal screenshot and accompanied by a concise technical explanation of what is happening inside the program.

The simulator is built on two core data structures: a singly-linked list (CommitList) that stores the commit history for each branch, and a FIFO queue (MergeQueue) that manages pending merge requests. Every commit node stores an ID, message, parent ID, and a full date+time timestamp.

The following 12 steps are covered in this walkthrough:

- Step 1 — VCS Initialization & Main Menu
- Step 2 — First Commit on 'main' (timestamp visible)
- Step 3 — Second Commit on 'main'
- Step 4 — Creating Branch 'dev' (deep copy)
- Step 5 — Switch to Branch 'dev'
- Step 6 — Commit on Feature Branch 'dev'
- Step 7 — Enqueue Merge Request (FIFO queue)
- Step 8 — Circular Reference Detection (merge aborted)
- Step 9 — Enqueue Valid Merge After New Commit
- Step 10 — Process Merge Successfully (FIFO)
- Step 11 — View Log (all branches with full history)
- Step 12 — Rollback to Commit & Program Exit

Step 1 — VCS Initialization & Main Menu

What happens:

When the program launches, the VCS constructor runs automatically. It creates the default branch 'main' by inserting a new CommitList into the branches hash map, then appends the very first CommitNode (ID: c001, message: 'Initial commit') to that list. The timestamp is recorded immediately using the repaired currentTime() function, which now outputs the full date and time.

Data structure: unordered_map<string, CommitList> branches — the 'main' key is inserted and its linked list is initialized with one node. commitCounter is set to 1 and incremented to 2 after this first commit.*

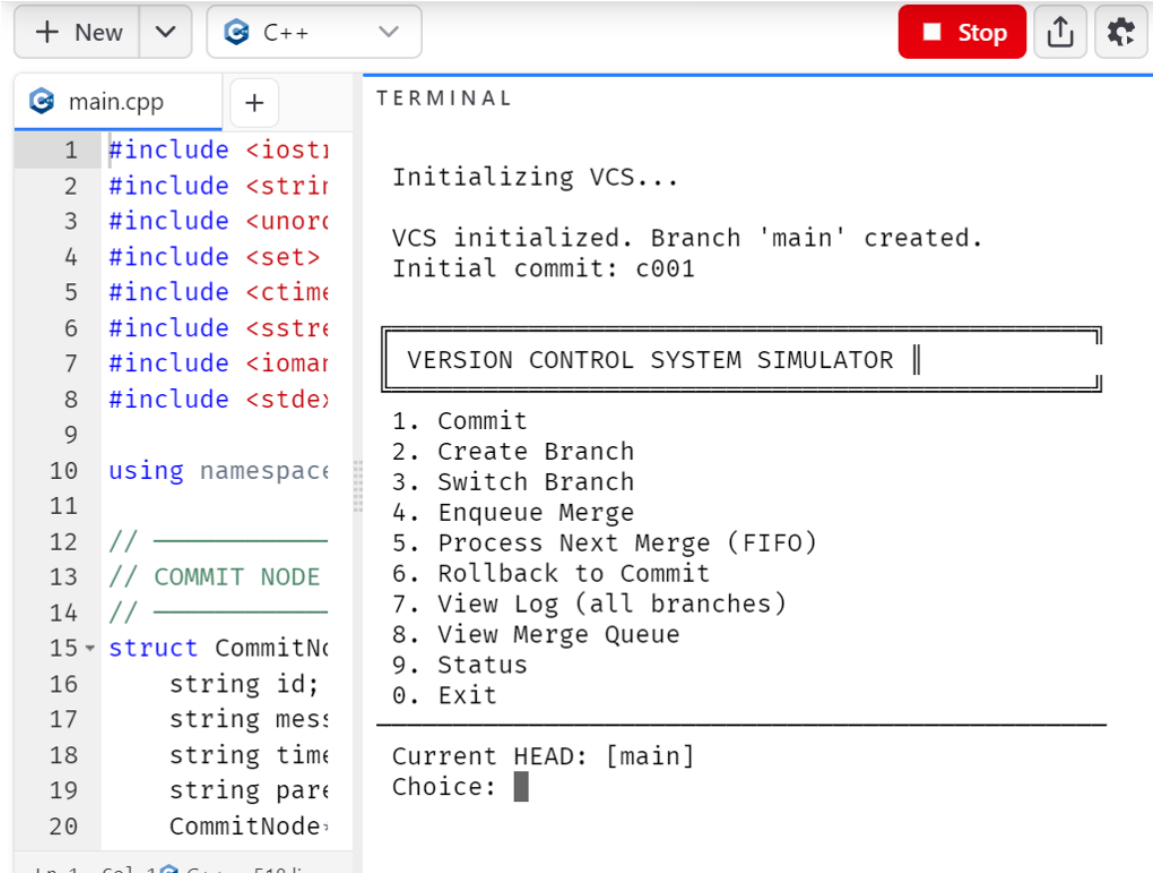


Figure 1 – Program starts: branch 'main' created, initial commit c001 appended, menu displayed.

Step 2 — First Commit on 'main'

What happens:

The user selects option 1 and enters the message 'initial setup'. The VCS calls commit(), which generates the next ID (c002), captures the current timestamp, and calls branches[currentBranch]->append(). This creates a new CommitNode on the heap and links it to the tail of the 'main' linked list. The parentId of c002 is automatically set to c001 (the previous tail).

Key fix visible here: the timestamp '2026-04-28 06:00:39' shows the repaired format %Y-%m-%d %H:%M:%S with buffer char[24], replacing the old time-only format.

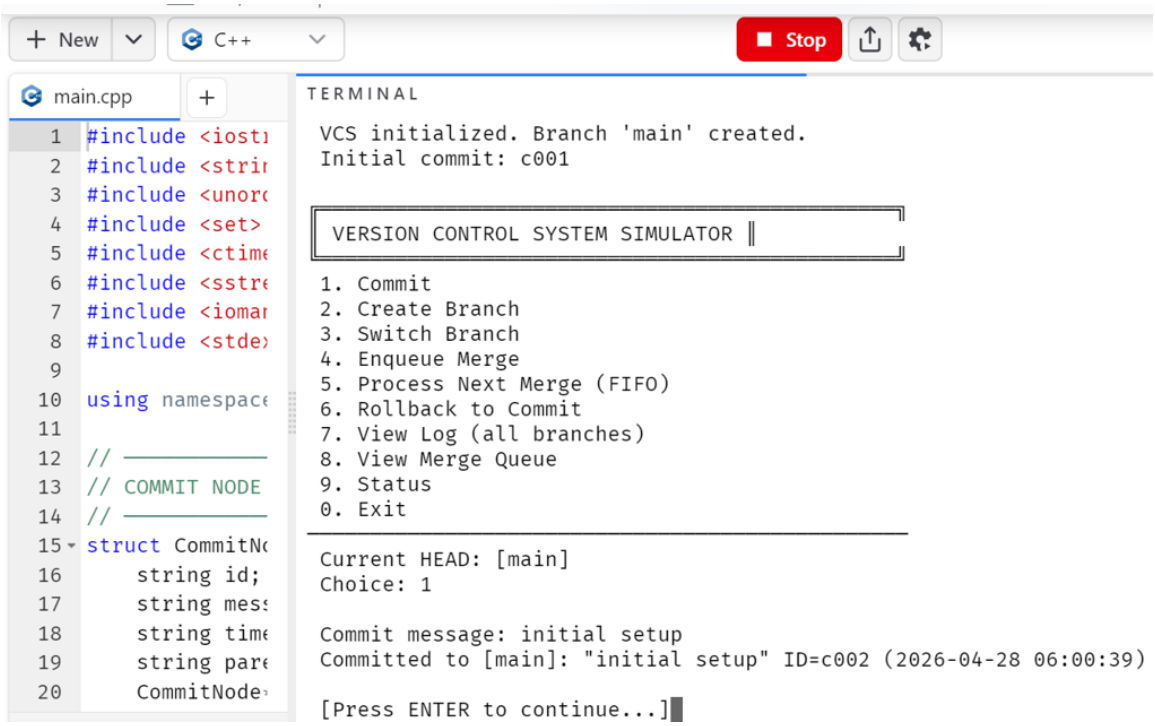


Figure 2 – Commit c002 'initial setup' appended to 'main'; date+time timestamp confirmed.

Step 3 — Second Commit on 'main'

What happens:

A second commit 'added login system' is made on 'main', producing node c003. The linked list for 'main' now contains three nodes: c001 -> c002 -> c003, with c003 as the new tail. Each node's next pointer forms the chain, and each node's parentId stores the ID of its predecessor, enabling the full history to be traversed in either direction.

> *At this point the 'main' CommitList has size = 3. The tail pointer is updated to c003 on every append, so the last commit is always O(1) to reach.*

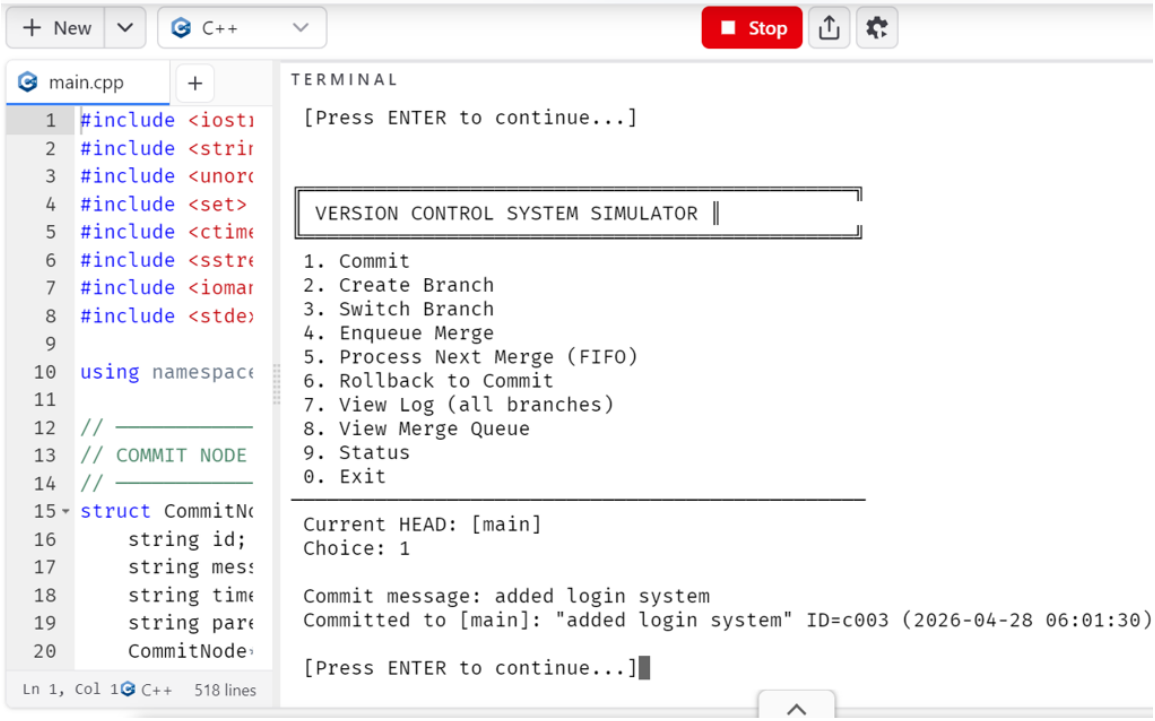


Figure 3 – Commit c003 'added login system' chained after c002; list now has 3 nodes.

Step 4 — Creating Branch 'dev'

What happens:

Option 2 creates a new branch named 'dev'. The VCS invokes the deep-copy constructor of CommitList, which walks the 'main' linked list node by node and creates brand-new CommitNode objects on the heap with identical content. The result is a completely independent list — modifying 'dev' will never affect 'main'.

> *Output confirms '3 commits copied'. The new entry branches['dev'] is inserted into the hash map pointing to this new independent list. Branch creation is O(n) where n is the number of commits in the source branch.*

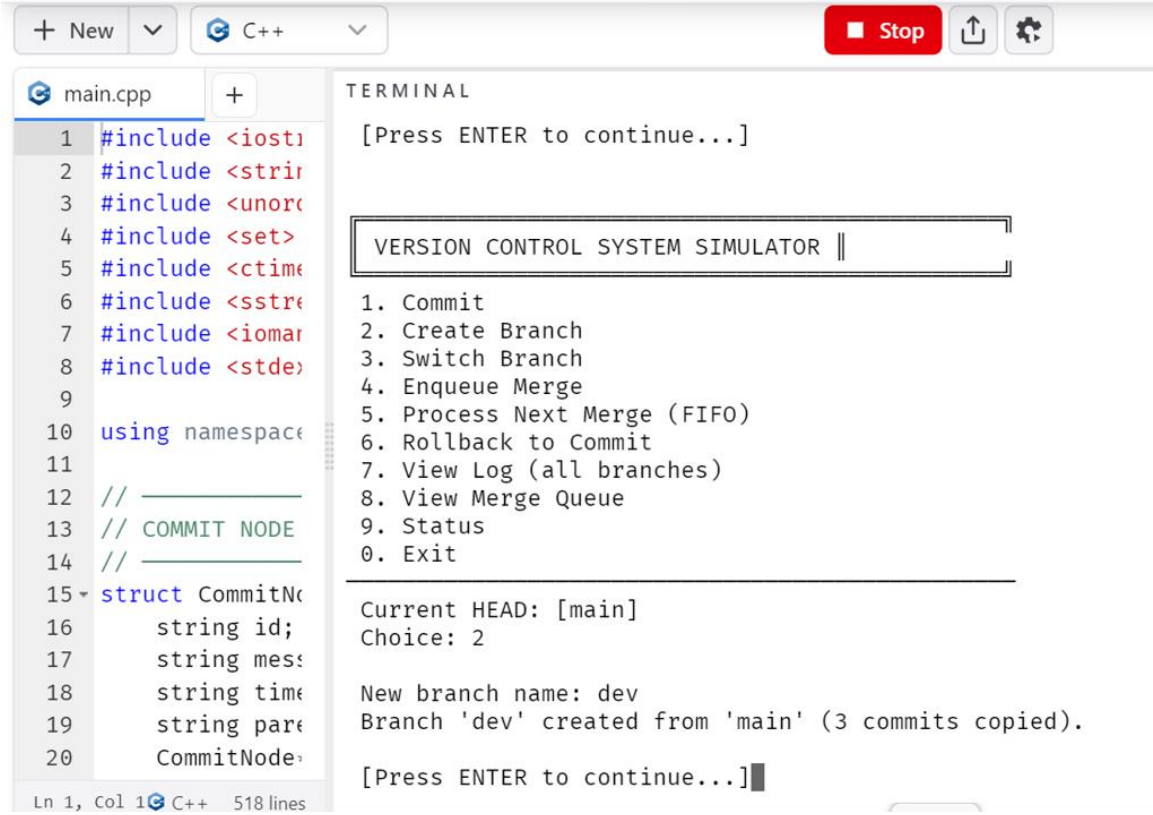


Figure 4 – Branch 'dev' deep-copied from 'main'; 3 commit nodes duplicated independently.

Step 5 — Switching to Branch 'dev'

What happens:

Option 3 updates the currentBranch string from 'main' to 'dev'. This is a simple pointer reassignment — no data is copied or moved. From this point on, all commit, rollback, and status operations act on the 'dev' linked list. The menu footer immediately reflects the change, showing 'Current HEAD: [dev]'.

> *switchBranch() first validates that the target branch exists in the hash map before reassigning currentBranch. Switching to a non-existent branch prints an error and leaves the current branch unchanged.*

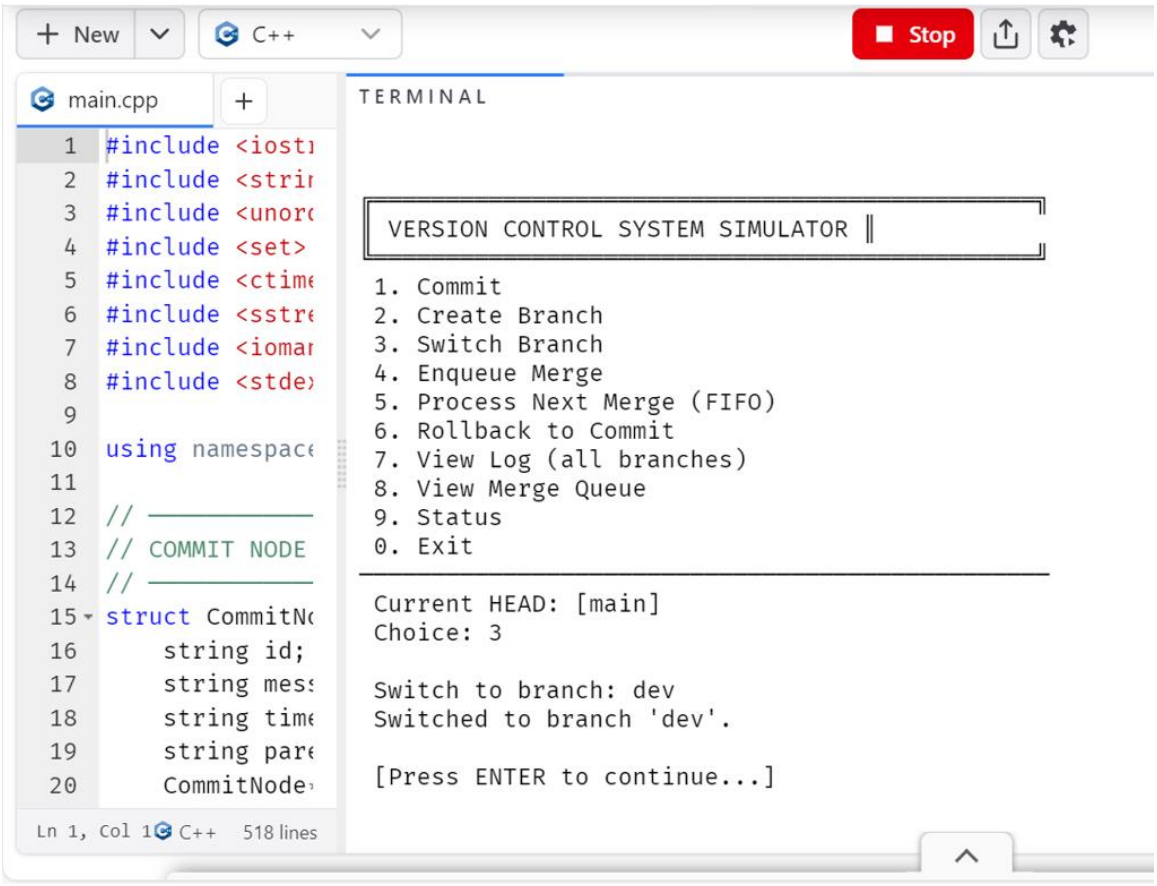


Figure 5 – currentBranch reassigned to 'dev'; all subsequent operations target 'dev'.

Step 6 — Committing on Feature Branch 'dev'

What happens:

With 'dev' as the active branch, the user commits 'implemented payment feature'. Node c004 is created and appended exclusively to the 'dev' linked list. The 'main' list still ends at c003. This commit is the unique change that will later be merged into 'main'.

> *After this step the commit histories diverge: main = [c001->c002->c003], dev = [c001->c002->c003->c004]. Only c004 is unique to 'dev'.*

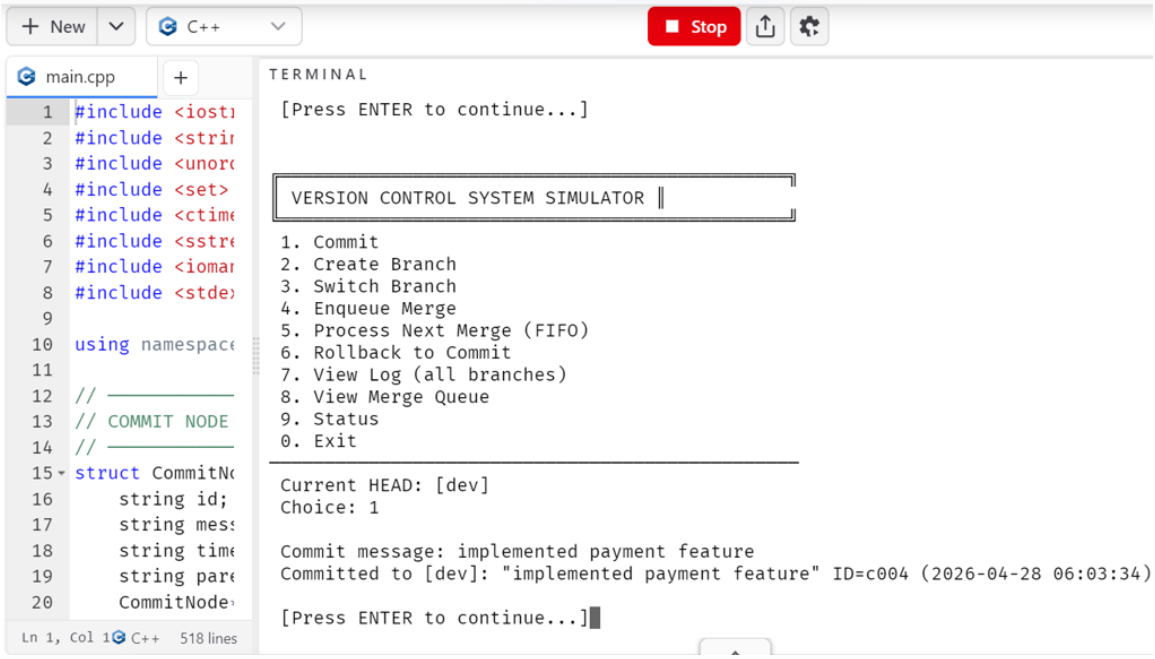


Figure 6 – Commit c004 'implemented payment feature' appended only to 'dev' at 06:03:34.

Step 7 — Enqueue Merge Request

What happens:

Back on 'main', the user enqueues a merge of 'dev' into 'main' using option 4. A new MergeRequest struct (sourceBranch='dev', targetBranch='main') is pushed to the rear of the MergeQueue. The queue size increments to 1. At this point no merge has been executed — the request is simply held in the FIFO queue awaiting processing.

> *The MergeQueue is a custom singly-linked queue. enqueue() creates a node on the heap and links it to rear->next, updating rear. Queue operations are O(1).*

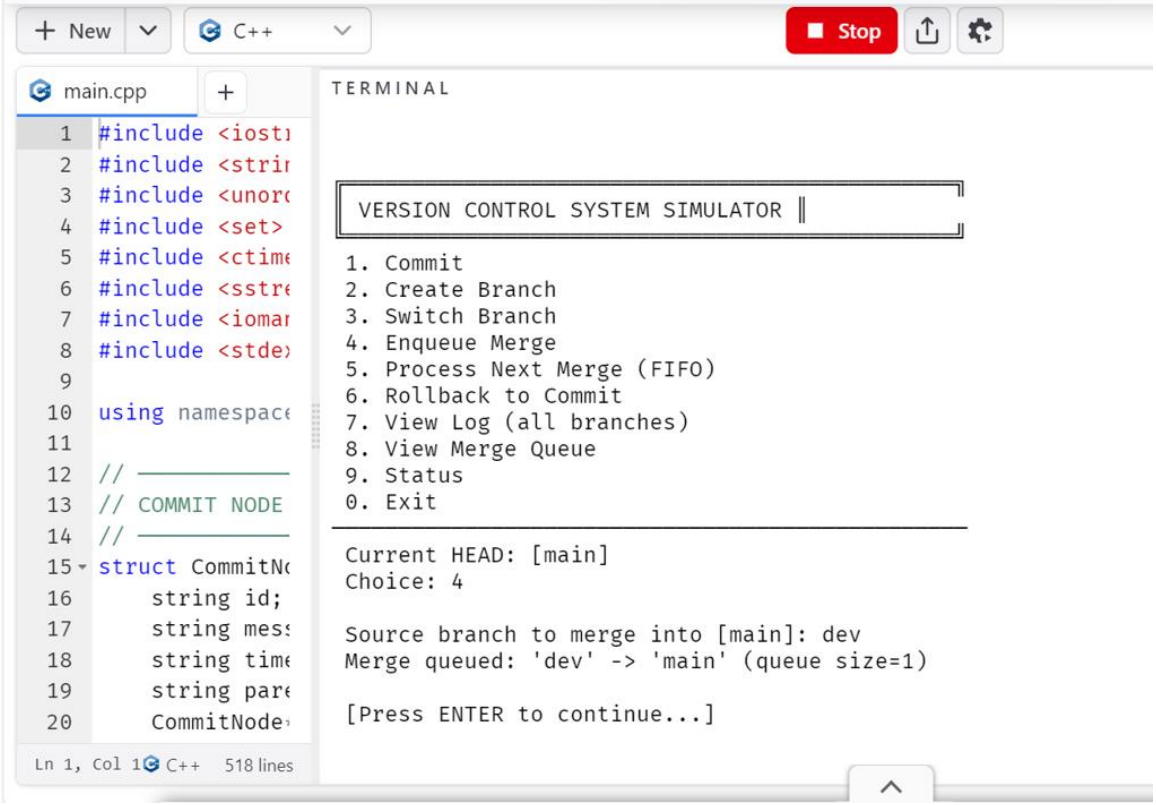


Figure 7 – MergeRequest 'dev' -> 'main' pushed to rear of FIFO queue; size = 1.

Step 8 — Circular Reference Detection (Merge Aborted)

What happens:

This screenshot shows the circular reference guard in action. When 'dev' was created directly from 'main' and no new commits existed on 'dev' yet, all of dev's commit IDs were already present in 'main'. The VCS collects all target IDs into a std::set, then scans each source commit — if every ID is found in the set, the merge is a no-op and is aborted with a clear warning message.

> *Guard logic: build set<string> targetIds from target list O(n), then check each source node O(m). If allPresent == true, print [WARN] and return without modifying the target. This prevents duplicate commits and detects redundant merges.*

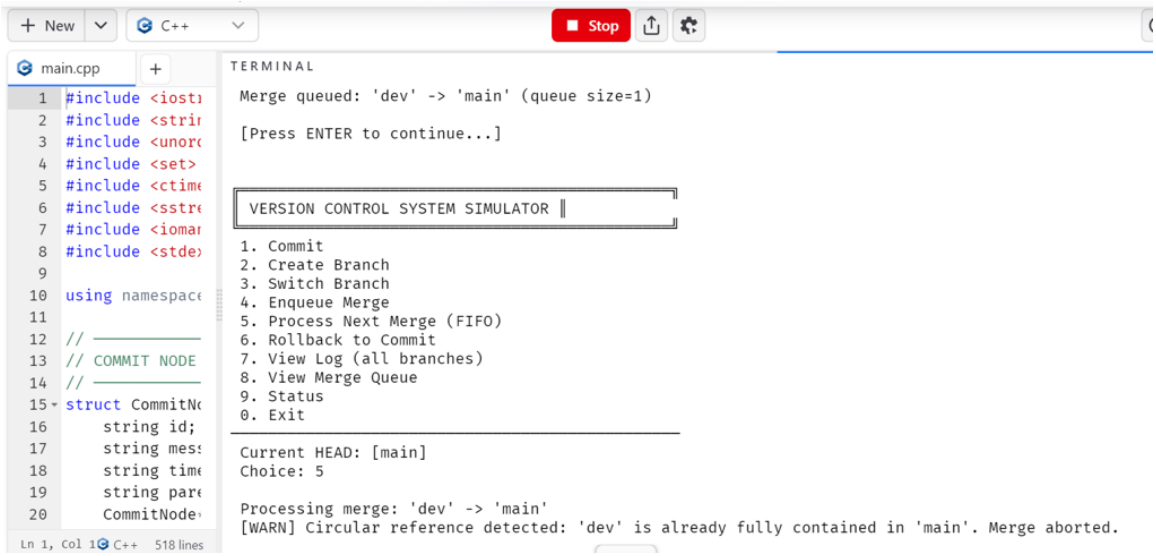


Figure 8 – Circular reference guard fires; dev's commits already fully contained in main.

Step 9 — Enqueue Valid Merge After New Commit

What happens:

After the circular reference abort, the user switches to 'dev', makes the new commit c004, then switches back to 'main' and enqueues a fresh merge request. This time 'dev' has a unique commit (c004) that does not exist in 'main', so the merge will succeed when processed. The new request is again pushed to the FIFO queue (size = 1).

> *This step demonstrates the correct workflow: enqueue after ensuring the source branch has diverged from the target. The queue does not validate at enqueue time — validation and execution happen together at processMerge().*

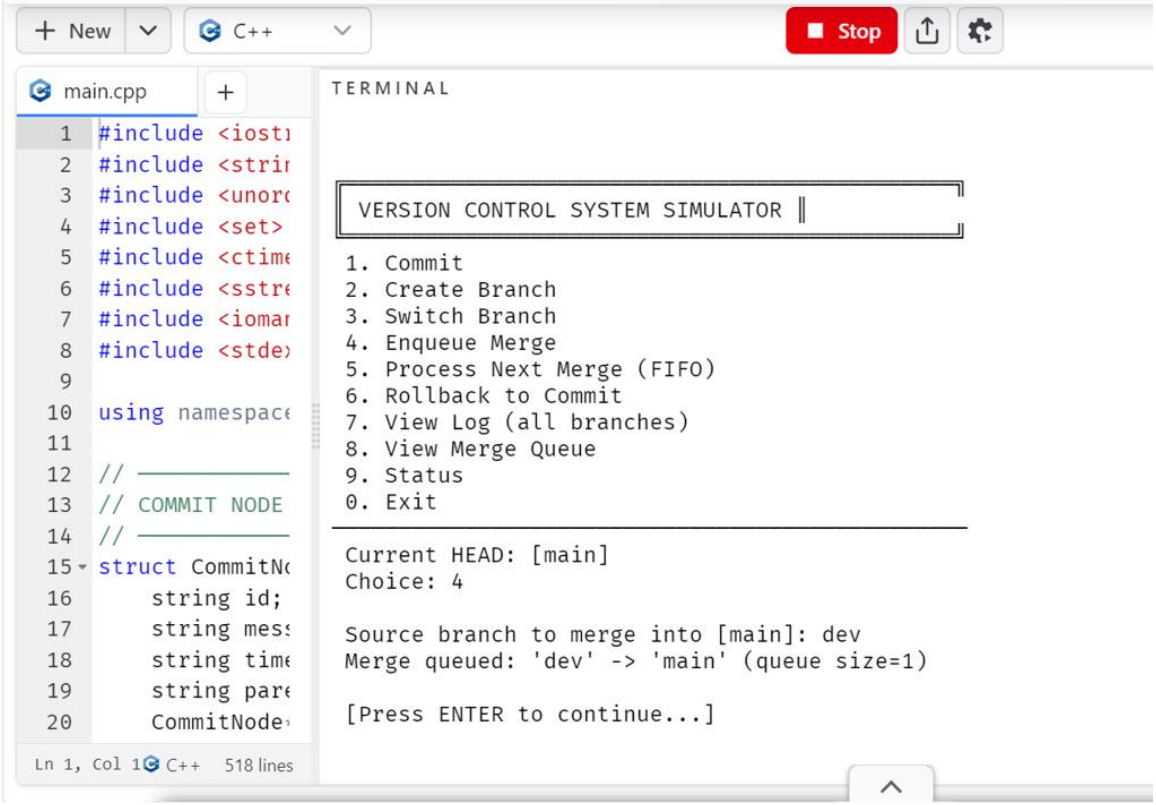


Figure 9 – Valid merge request queued: 'dev' -> 'main' (queue size = 1, c004 unique in dev).

Step 10 — Processing the Merge (FIFO)

What happens:

Option 5 dequeues the front MergeRequest and executes it. The engine builds a set of all target commit IDs, then walks the source list to find commits not in that set. Each unique commit is loaded into an internal FIFO conflict queue and then dequeued in order and appended to the target list with a '[merged]' prefix on the message. c004 is the only unique commit, so exactly 1 commit is applied.

> *Merge applies commits in source order (FIFO), preserving history sequence. The merged node gets a fresh timestamp at the moment of merge, not the original commit timestamp. After processing, the MergeQueue size returns to 0.*

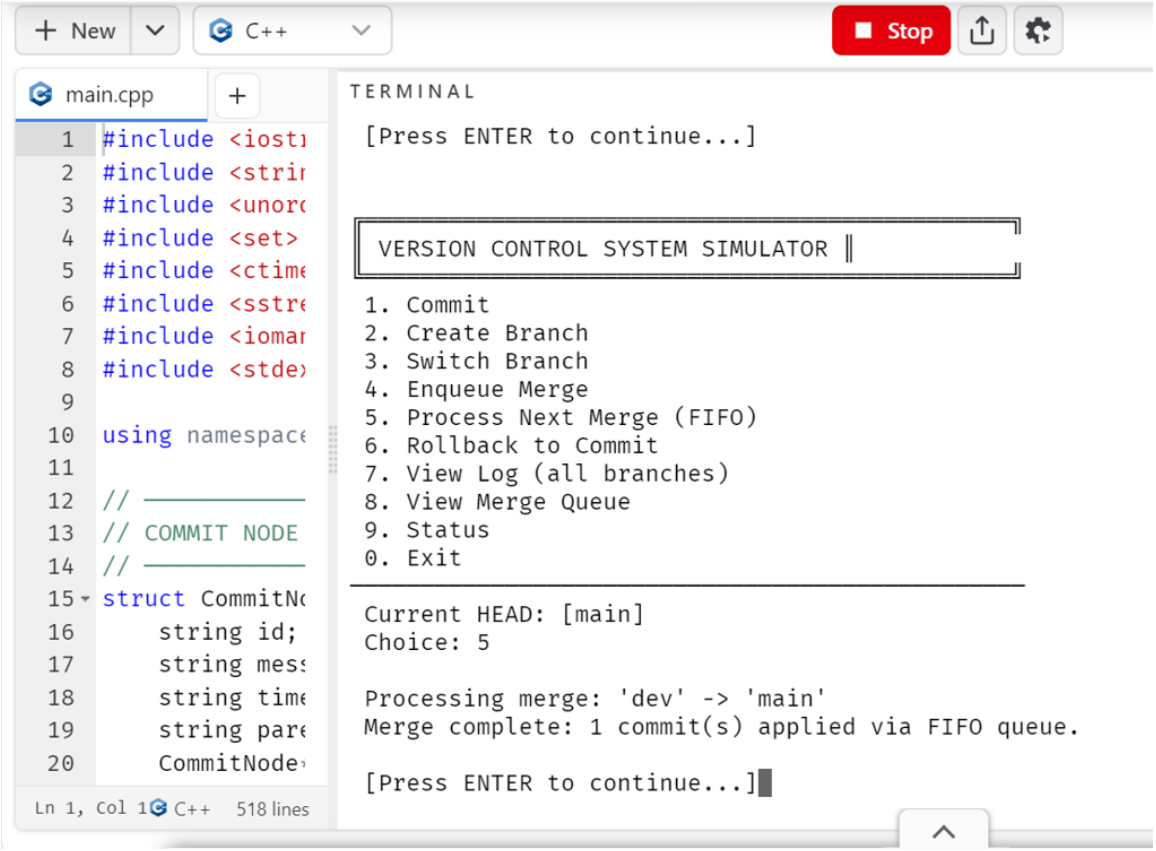


Figure 10 – Merge complete: 1 commit (c004) applied to 'main' via FIFO queue.

Step 11 — View Log (All Branches)

What happens:

Option 7 iterates over every entry in the branches hash map and calls print() on each CommitList. For each node it outputs the index, commit ID, message in quotes, and the full date+time timestamp. The active branch is marked with '<-- HEAD'. The log clearly shows both branches side by side, confirming that c004 now exists in 'main' as a merged commit while 'dev' retains its original c004 as the original commit.

> *The log for 'main' ends with c004 '[merged] implemented payment feature' (2026-04-28 06:04:48) [HEAD], proving the merge was applied with a new timestamp. The 'dev' log ends with c004 'implemented payment feature' (2026-04-28 06:03:34) [HEAD], the original unmodified node.*

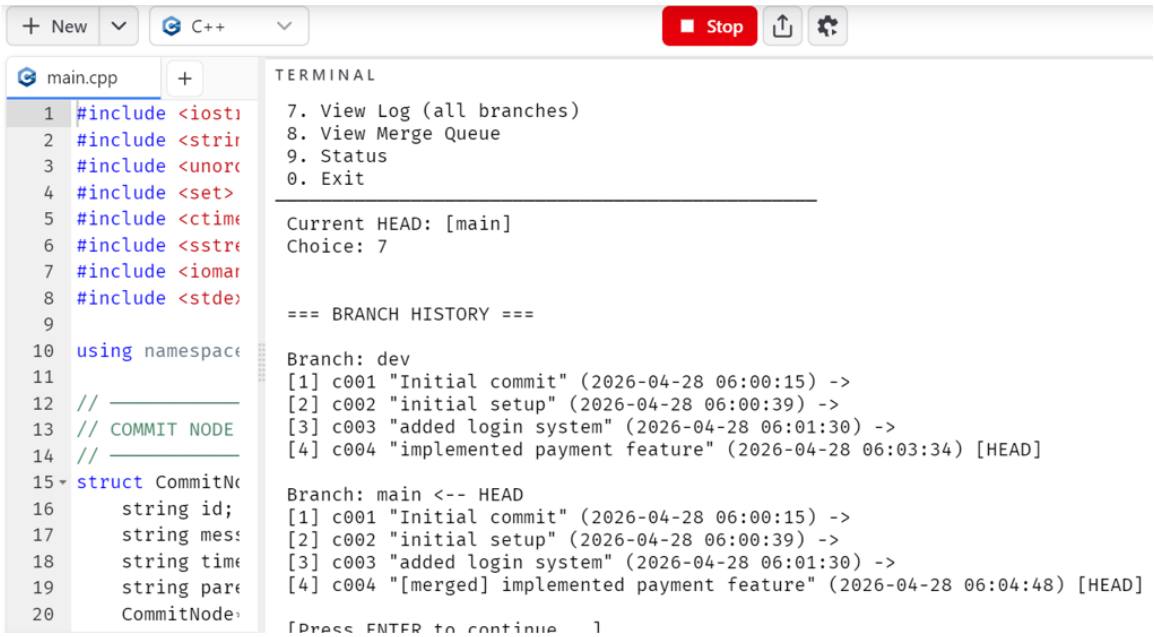


Figure 11 – Branch history log: both branches shown with full commit chains and timestamps.

Step 12 — Rollback to Commit & Program Exit

What happens (Rollback):

Option 6 triggers a rollback on the current branch. The user specifies commit ID 'c002'. The VCS walks the 'main' linked list until it finds the node with that ID, then deletes every node after it (freeing heap memory one by one) and sets tail = c002 and c002->next = nullptr. c003 and the merged c004 are permanently discarded from 'main'. The list is now c001 -> c002 [HEAD].

> *rollbackTo() is O(n) for finding the target node, then O(k) for deleting k subsequent nodes. The size counter is decremented for each deleted node. This operation is irreversible within the same session — there is no redo stack.*

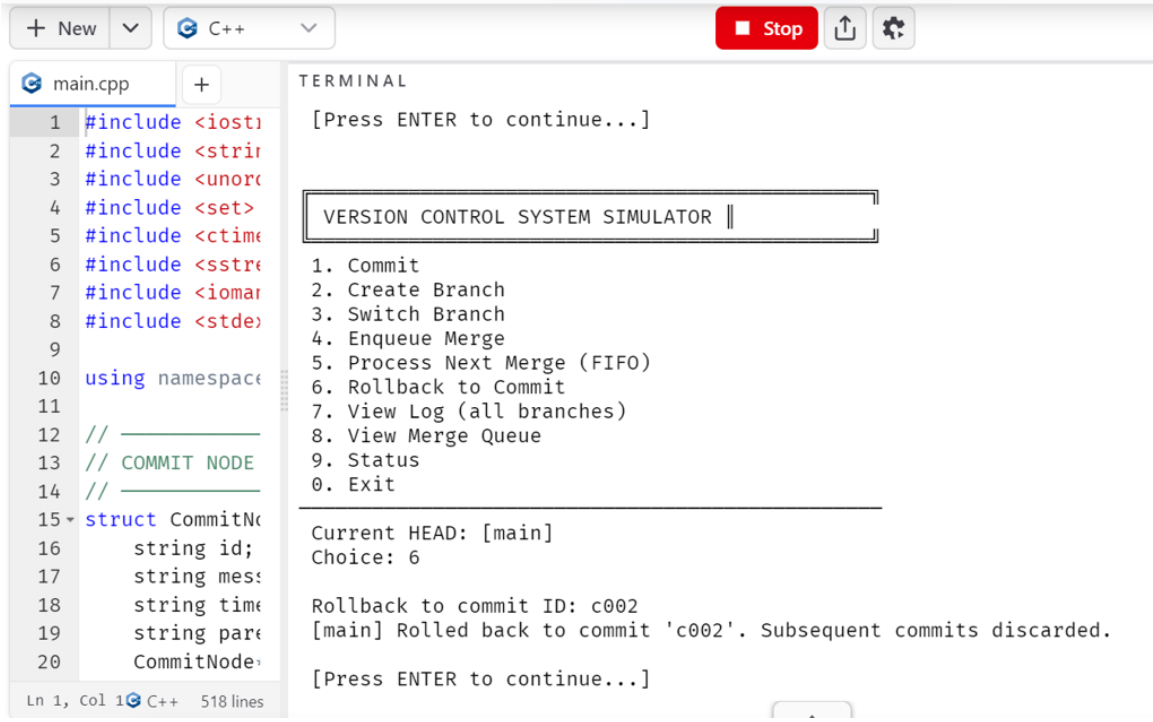


Figure 12a – Rollback to c002 on 'main'; all subsequent commits (c003, merged c004) discarded.

What happens (Exit):

Selecting option 0 breaks the main do-while loop and prints the goodbye message. The VCS destructor then runs automatically, iterating over every entry in the branches hash map and calling delete on each CommitList pointer. Each CommitList destructor walks its linked list and frees every CommitNode. The MergeQueue destructor frees any remaining queued requests. The program exits cleanly with return code 0.

> *Return code 0 visible in the terminal confirms no runtime errors, no memory leaks from unfreed heap allocations, and clean destructor execution throughout.*

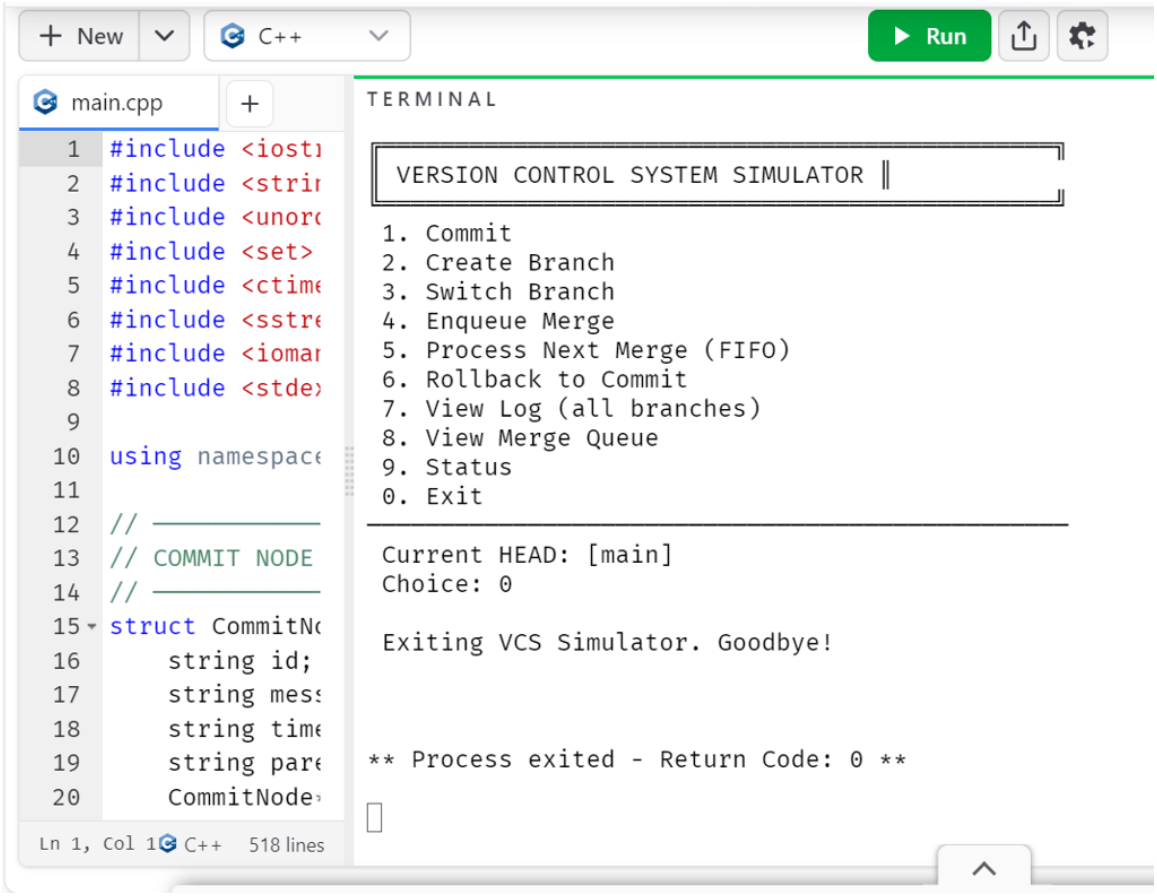


Figure 12b – Program exit: destructor chain frees all heap memory; return code 0.

Summary Table

The table below summarises all 12 steps, the menu option used, the data structure involved, and the observed result.

Step	Operation	Menu	Data Structure	Outcome
1	VCS Init	0 (auto)	Linked List (CommitList)	Branch 'main' + c001 created automatically
2	First Commit (c002)	1	Linked List append	Node appended; date+time timestamp shown
3	Second Commit (c003)	1	Linked List append	3-node chain: c001->c002->c003 on 'main'
4	Create Branch 'dev'	2	Deep-copy CommitList	3 nodes deep-copied to independent 'dev' list
5	Switch to 'dev'	3	Branch pointer update	currentBranch = 'dev'; HEAD updated
6	Commit on 'dev' (c004)	1	Linked List append	c004 unique to 'dev'; histories now diverge
7	Enqueue Merge	4	FIFO MergeQueue push	MergeRequest pushed to rear; queue size = 1
8	Circular Ref (abort)	5	std::set + FIFO queue	[WARN] all source IDs in target; merge aborted
9	Enqueue Valid Merge	4	FIFO MergeQueue push	Valid request queued after c004 committed
10	Process Merge	5	FIFO queue dequeue+append	c004 merged into 'main' via FIFO; 1 commit
11	View Log	7	Linked List traversal	Both branches printed with full timestamps
12	Rollback + Exit	6 / 0	Linked List truncation	c002 set as tail; c003+c004 freed; exit code 0